

GRANJA CAFETERA

Documentación Técnica del Proyecto

Giovanni Andrés Torres Pinzón

Motor de Juego: Godot Engine 4 Lenguaje: GDScript
Proyecto: gv_start_project

Mayo 2026



1. Descripción General del Proyecto

Granja Cafetera es un videojuego 2D de simulación agrícola desarrollado en Godot Engine 4 con GDScript. El jugador administra una granja dedicada exclusivamente al cultivo y comercialización de café. La mecánica central gira en torno al ciclo de vida del café: plantar, cultivar, cosechar y vender al NPC vendedor presente en el nivel.

1.1 Objetivo del Juego

- Plantar semillas de café usando la herramienta Hoe (azadón).
- Cuidar los cultivos mientras crecen en varios ticks de tiempo.
- Cosechar el café maduro con la herramienta Axe (hoz) antes de que muera.
- Vender el café cosechado al NPC Vendedor para obtener monedas (Item.WOOD).
- Comprar más semillas de café con las monedas acumuladas.

1.2 Estructura General del Proyecto

Carpeta / Archivo	Descripción
<code>gv_start_project/</code>	Raíz del proyecto Godot
<code>global/</code>	Scripts Autoload: Enum, Data, Inventory
<code>scenes/levels/</code>	Escenas y scripts del nivel de juego
<code>scenes/characters/</code>	Scripts de personajes (jugador, NPC)
<code>graphics/</code>	Sprites, tilesets, iconos
<code>.godot/</code>	Caché interna del editor (no editar)

2. Scripts Globales (Autoload)

Los siguientes scripts se cargan automáticamente al iniciar el juego y son accesibles desde cualquier escena sin necesidad de importación.

2.1 enums.gd — Definición de Tipos

Contiene todas las enumeraciones del juego que sirven como identificadores de tipo para herramientas, semillas e ítems.

Enum	Valores
Tool	AXE, HOE, SWORD, WATER, FISH, SEED
Seed	TOMATO, CORN, PUMPKIN, WHEAT, COFFEE
Item	WOOD (monedas), APPLE, TOMATO, CORN, WHEAT, PUMPKIN, FISH, COFFEE
Style	BASIC, COWBOY, ENGLISH, BASEBALL, BEANIE, STRAW, CAP
Machine	SPRINKLER, FISHER, SCARECROW, DELETE
Shop	MAIN, HAT

Nota importante

Item.WOOD actúa como la moneda del juego. Aunque el nombre sugiere 'madera', en la implementación actual funciona exclusivamente como unidad monetaria. Las transacciones de compra/venta con el NPC se expresan en esta unidad.

2.2 data.gd — Datos Estáticos del Juego

Almacena configuración estática: texturas de herramientas, datos de plantas y skins del jugador.

Constante	Contenido
PLAYER_SKINS	Textura por cada estilo de personaje (6 variantes)
PLANT_DATA	Configuración de cada cultivo: textura, velocidad, frames, recompensa
TOOL_TEXTURES	Icono gráfico de cada herramienta para mostrar en el HUD
TOOL_STATE_ANIMATIONS	Nombre de la animación del jugador por herramienta
TILE_SIZE	Tamaño de tile en píxeles (16 px)

Datos del cultivo de Café (Enum.Seed.COFFEE)

Parámetro	Valor
texture	res://graphics/plants/coffee.png
h_frames	4 frames (semilla → creciendo → crecido → maduro)
grow_speed	0.8 — crece en ~6 ticks (cada 2 segundos)
death_max	4 ticks después de madurar antes de morir
reward	Enum.Item.COFFEE (1 café al inventario) + 8 monedas

2.3 inventory.gd — Sistema de Inventario

Gestiona el inventario del jugador: semillas, ítems y monedas. Funciones principales:

- `get_item(type)` — Retorna la cantidad de un ítem en el inventario.
 - `add_item(type, amount)` — Suma cantidad de un ítem.
 - `consume_item(type, amount)` — Resta cantidad de un ítem.
 - `add_seed(type, amount)` — Suma semillas de un tipo.
 - `consume_seed(type)` — Consume una semilla (retorna false si no hay).
 - `spend(cost_dict)` — Descuenta un diccionario de costos; retorna false si no alcanza.
-

3. Sistema de Movimiento del Jugador

El jugador está representado por un nodo `CharacterBody2D` con un sistema de movimiento en 8 direcciones. El script `player.gd` (en `scenes/levels/`) controla todo el comportamiento.

3.1 Controles de Movimiento

Acción (InputMap)	Tecla por defecto
right	D / Flecha Derecha
left	A / Flecha Izquierda
up	W / Flecha Arriba
down	S / Flecha Abajo
action	Espacio (SPACE) — usar herramienta / interactuar con NPC
tool_forward	E — siguiente herramienta
tool_backward	Q — herramienta anterior

3.2 Lógica de Movimiento (`_physics_process`)

El movimiento se ejecuta en cada frame de física. El flujo es el siguiente:

Flujo de <code>_physics_process</code>
1. Se lee el input direccional (right, left, up, down).
2. Si no está usando una herramienta: reproduce animación walk o idle.
3. Si se presiona <code>tool_forward/tool_backward</code> : cambia la herramienta activa.
4. Si se presiona <code>action</code> (SPACE): llama a <code>_use_current_tool()</code> .

5. `velocity = direction * SPEED (80 px/s)`.
6. `move_and_slide()` aplica el movimiento con detección de colisiones.
7. La posición se clampea dentro de los límites del mapa (`farm_min / farm_max`).

3.3 Límites del Mapa

Los límites se calculan dinámicamente en `level.gd` al arrancar la escena, leyendo el `rect` del `TileMapLayer` del suelo y convirtiéndolo a coordenadas mundiales. Se pasan al jugador con:

```
player.set_farm_bounds(world_min, world_max)
```

El jugador usa `clamp()` en `position.x` e `position.y` para nunca salir del área.

3.4 Sistema de Animaciones

Las animaciones siguen el patrón: `[accion]_[direccion]`. Por ejemplo:

- `idle_down`, `idle_up`, `idle_left`, `idle_right`
- `walk_down`, `walk_up`, `walk_left`, `walk_right`
- `hoe_down`, `hoe_up`, `hoe_left`, `hoe_right` — animación al usar Hoe
- `axe_down`, `axe_up`, `axe_left`, `axe_right` — animación al usar Axe (hoz)

La variable `last_dir` registra la última dirección para seleccionar la animación correcta incluso cuando el jugador está estático.

4. Herramienta Hoe (Azadón) — Mecánica Principal

La herramienta Hoe (`Enum.Tool.HOE`) es la herramienta central del juego. En una sola acción realiza tres pasos simultáneos: arar el suelo, regar los tiles y sembrar semillas de café.

HOE — Resumen de Acción

Presionar `SPACE` con Hoe seleccionada:

- Paso 1: Ara un área 3x3 centrada en el jugador (crea tiles de suelo).
- Paso 2: Riega automáticamente todos los tiles arados.
- Paso 3: Siembra semillas de café en cada tile regado (consume del inventario).

4.1 Flujo Detallado de `_use_hoe()`

Al presionar `SPACE` con Hoe, se emite la señal `tool_used` desde el jugador. `Level.gd` la recibe y ejecuta `_use_hoe(tile_pos)`:

Paso	Descripción del código
1. Iterar 3x3	Bucle doble dy/dx de -1 a +1 alrededor del tile del jugador.
2. Crear tile	Si el tile no existe en tile_data: set_cell() en soil_layer con SOIL_ATLAS_COORD.
3. Registrar dato	tile_data[t] = { 'watered': false, 'plant': null }
4. Regar	set_cell() con WATER_ATLAS_COORD y tile_data[t]['watered'] = true
5. Sembrar	Si tile regado y sin planta: consume_seed(COFFEE) y llama _plantar_en(t)
6. Instanciar	_plantar_en() instancia PlantScene y llama setup(Enum.Seed.COFFEE)

4.2 Animación del Jugador al Usar Hoe

Al presionar SPACE, el método `_use_current_tool()` en `player.gd`:

- Establece `is_using_tool = true` (bloquea movimiento durante la animación).
- Reproduce la animación `hoe_[direccion]` (ej: `hoe_down`).
- Espera `await anim.animation_finished`.
- Emite `tool_used` — `level.gd` procesa la acción.
- Restaura `is_using_tool = false` y vuelve a idle.

Importante — Sin semillas

Si el inventario no tiene semillas de café, el paso de siembra se omite.

El jugador debe comprar semillas al NPC vendedor antes de usar el Hoe.

Mensaje en consola: '[HOE] Sin semillas — compra más al NPC.'

5. Ciclo de Vida de las Plantas

Cada planta es una instancia de `plant.tscn` / `plant.gd` (Node2D) con un Timer interno que avanza su estado cada 2 segundos.



5.1 Estados de la Planta

Estado	Descripción
SEED	Recién plantada. Sprite frame 0.
GROWING	Creciendo. Frame avanza progresivamente de 0 a h_frames-2.
READY	Lista para cosechar. Frame = h_frames-1 (último). Pulsa/parpadea.
DEAD	No fue cosechada a tiempo. Se vuelve gris y desaparece.

5.2 Progresión por Ticks

Con `grow_speed = 0.8` para el café, `max_grow_ticks = max(2, int(5.0 / 0.8)) = 6` ticks (~12 segundos reales). Después de READY, tiene `death_max = 4` ticks (~8 segundos) para ser cosechada.

5.3 Cosecha con Axe (Hoz)

Al usar la herramienta Axe (Enum.Tool.AXE) sobre una planta en estado READY:

- Se detiene la animación pulse.
- Se reproduce la animación `coins_popup` (+8 monedas flotando).
- `Inventory.add_item(Item.WOOD, 8)` — agrega 8 monedas.
- `Inventory.add_item(Item.COFFEE, 1)` — agrega 1 café al inventario.
- La planta notifica al level para liberar el tile.
- `queue_free()` elimina el nodo de la escena.

6. NPC Vendedor — Detección y Panel de Tienda

El NPC Vendedor es un `CharacterBody2D` (`npc_vendedor.gd`) que gestiona la interacción de compra/venta con el jugador mediante un sistema de detección de colisión y panel de UI.

6.1 Sistema de Detección de Colisión (Area2D)

El NPC tiene un nodo hijo Area2D llamado DetectionArea con una CollisionShape2D circular. Este detector maneja la proximidad del jugador:

Señal del Area2D	Acción ejecutada
body_entered(body)	Si body está en grupo 'player': player_nearby = true, muestra hint de interacción
body_exited(body)	Si body está en grupo 'player': player_nearby = false, oculta hint, cierra tienda si abierta

Flujo de interacción con SPACE

1. El jugador camina hacia el NPC — entra en el Area2D.
2. player_nearby = true → aparece un indicador visual ('Presiona SPACE').
3. El jugador presiona SPACE (tecla 'action').
4. _input() detecta is_action_just_pressed('action') con player_nearby == true.
5. Se llama _open_shop() → dialog_panel.visible = true.
6. El panel superior muestra las secciones COMPRAR y VENDER.
7. Al presionar SPACE de nuevo (o al alejarse): se llama _close_shop().

6.2 Panel de Tienda (shop_panel.gd)

El ShopPanel (PanelContainer) se registra en el grupo 'dialog_panel'. Al abrirse con open_shop(data), construye dinámicamente dos secciones:

Sección del Panel	Contenido
VENDER (SellList)	Muestra el botón para vender todo el café del inventario. Precio: 6 monedas por unidad.
COMPRAR (BuyList)	Muestra el botón para comprar 5 semillas de café. Costo: 5 monedas.
Botón Cerrar	Cierra el panel y oculta la UI (dialog_panel.visible = false).

Lógica de Venta (jugador vende café)

- Lee Inventory.get_item(Item.COFFEE) para saber cuánto café tiene.
- Si amount > 0: muestra 'Vender Café xN → N*6 monedas'.
- Al confirmar: consume_item(COFFEE, amount) y add_item(WOOD, amount*6).
- El panel se reconstruye inmediatamente para reflejar el nuevo estado.

Lógica de Compra (jugador compra semillas)

- Lee `Inventory.get_item(Item.WOOD)` para verificar fondos.
- Muestra precio: 5 monedas por 5 semillas.
- Si `monedas < 5`: el botón aparece deshabilitado (`btn.disabled = true`).
- Al confirmar: `spend({WOOD: 5})` descuenta monedas y `add_seed(COFFEE, 5)`.

6.3 Datos de la Tienda (`_get_shop_data`)

El NPC define los precios en el método `_get_shop_data()` que devuelve un diccionario:

```
seeds:          { COFFEE: { price: {WOOD: 5}, name: 'Semilla de Café' } }  
buy_prices:     { COFFEE: 6 }      # monedas que paga por cada café cosechado
```

7. HUD — Panel Superior de Estado

El panel superior (`hud_status.gd`) muestra información contextual al jugador en tiempo real. Se actualiza en cada frame desde `level.gd/_process()`.

Mensaje mostrado	Condición
 Arado — usa HOE para sembrar	Tile arado pero no regado
 Regado — usa Semilla para plantar	Tile regado pero sin planta
 Creciendo... espera	Tile con planta en estado GROWING
 LISTO — presiona AXE para cosechar (+8 monedas)	Planta en estado READY
(vacío)	Jugador en tile sin datos

Además, la toolbar (`tool_bar.gd`) en la parte superior resalta en amarillo la herramienta actualmente seleccionada.

8. Árbol de Escenas

La escena principal del juego (level.tscn) tiene la siguiente estructura de nodos:

Árbol de nodos — level.tscn	
Level (Node2D) [level.gd]	
├─	Layers (Node2D)
──	Ground (TileMapLayer) — suelo base visible
──	Soil (TileMapLayer) — tiles de tierra arada/regada
├─	Objects (Node2D)
──	Player (CharacterBody2D) [player.gd]
──	Animation / AnimationPlayer
──	CollisionShape2D
──	NPC_Vendedor (CharacterBody2D) [npc_vendedor.gd]
──	Sprite2D
──	DetectionArea (Area2D)
──	CollisionShape2D
├─	UI (CanvasLayer)
──	HUDStatus [hud_status.gd] — grupo 'hud_status'
──	ToolBar [tool_bar.gd] — grupo 'toolbar'
──	ShopPanel [shop_panel.gd] — grupo 'dialog_panel'
──	InteractHint (Label) — grupo 'interact_hint'

9. Flujo de Señales

Godot usa señales para comunicar eventos entre nodos desacoplados. Las principales señales del proyecto son:

Señal	Descripción del flujo
Player.tool_used	Emitida al finalizar la animación de herramienta → recibida por Level._on_tool_used()
NPCVendedor.shop_opened	Emitida al abrir la tienda → puede usarse para pausar al jugador

<code>NPCVendedor.shop_closed</code>	Emitida al cerrar la tienda
<code>ShopPanel.closed</code>	Emitida al presionar 'Cerrar' en el panel
<code>Area2D.body_entered</code>	Detecta cuando el jugador entra al radio del NPC
<code>Area2D.body_exited</code>	Detecta cuando el jugador sale del radio del NPC
<code>Timer.timeout (en Plant)</code>	Avanza el estado de la planta cada 2 segundos
<code>AnimationPlayer.animation_finished</code>	Usado con await para secuenciar animaciones

10. Ciclo Completo de Jugabilidad

El flujo de juego de punta a punta sigue estos pasos:

Ciclo de Juego Completo

① INICIO: El jugador comienza con semillas de café (parcela inicial pre-sembrada).

② PLANTAR con HOE:

- Seleccionar Hoe en la toolbar (primer slot, resaltado en amarillo).
- Caminar a un área vacía de la granja.
- Presionar SPACE → animación hoe_[dir] → el área 3x3 se ara, riega y siembra.

③ ESPERAR CRECIMIENTO:

- Las plantas avanzan: SEED → GROWING → READY (~12 segundos).
- El HUD muestra el estado del tile bajo el jugador.

④ COSECHAR con AXE:

- Cuando el HUD muestra '☉ LISTO', seleccionar Axe.
- Presionar SPACE → animación axe_[dir] → +8 monedas + 1 café al inventario.
- La planta desaparece y el tile queda libre.

⑤ VENDER al NPC:

- Caminar hacia el NPC → detector Area2D activa el hint de interacción.
- Presionar SPACE → panel superior aparece con secciones COMPRAR / VENDER.
- Hacer clic en 'Vender Café' → recibir monedas (6 por unidad).

⑥ COMPRAR SEMILLAS:

- Con monedas suficientes, clic en 'Comprar Semilla de Café (x5) — 5 monedas'.
- Cerrar panel → repetir desde ②.

11. Glosario Técnico

Término	Definición
CharacterBody2D	Nodo de Godot para personajes con movimiento y colisiones físicas.
TileMapLayer	Nodo para renderizar mapas de tiles (suelo, obstáculos).
Area2D	Detector de colisiones sin física; genera señales <code>body_entered/exited</code> .
GDScript	Lenguaje de scripting propio de Godot, similar a Python.
Autoload	Script cargado globalmente al inicio; accesible desde cualquier escena.
tile_data	Diccionario en <code>level.gd</code> que registra el estado (regado, planta) de cada tile.
farm_min/max	Vectores que definen los límites del mapa; el jugador no puede salir.
is_using_tool	Flag booleano que bloquea movimiento y acciones durante una animación.
last_dir	Almacena la última dirección del jugador para seleccionar la animación.
emit_signal	Función de Godot para disparar una señal y notificar a los escuchas.
queue_free	Elimina el nodo de la escena de forma segura al final del frame.
await	Suspende la ejecución hasta que una señal se emita (GDScript <code>async</code>).
HOE	Herramienta multipropósito: ara + riega + siembra en un solo uso.
AXE	Herramienta de cosecha; usada sobre plantas maduras para recolectarlas.